

Desenvolvimento Back-End



Aula 01

- Apresentação da Disciplina;
- Introdução a OO
 - Classes
 - Métodos
 - Atributos
 - Objetos / Instâncias

Prof. MSc. Luiz Carlos Machi Lozano

luiz.machi@sp.senai.br

- ✓ Bacharel em Sistemas de Informação;
- ✓ Mestre em Engenharia de Produção com ênfase em desenvolvimento de sistemas inteligentes;
- ✓ Pesquisador nas Áreas
 - ❑ Inteligência Artificial
 - ❑ Ciência de Dados
 - ❑ Machine Learning
 - ❑ Engenharia de Software



Conteúdo programático

- **Versionamento**

- Definição
- Repositório
- Software

- **Bibliotecas**

- Matemáticas
- Randômicas
- Cadeia de caracteres (strings)

- **Testes**

- Unitários
- Integrados

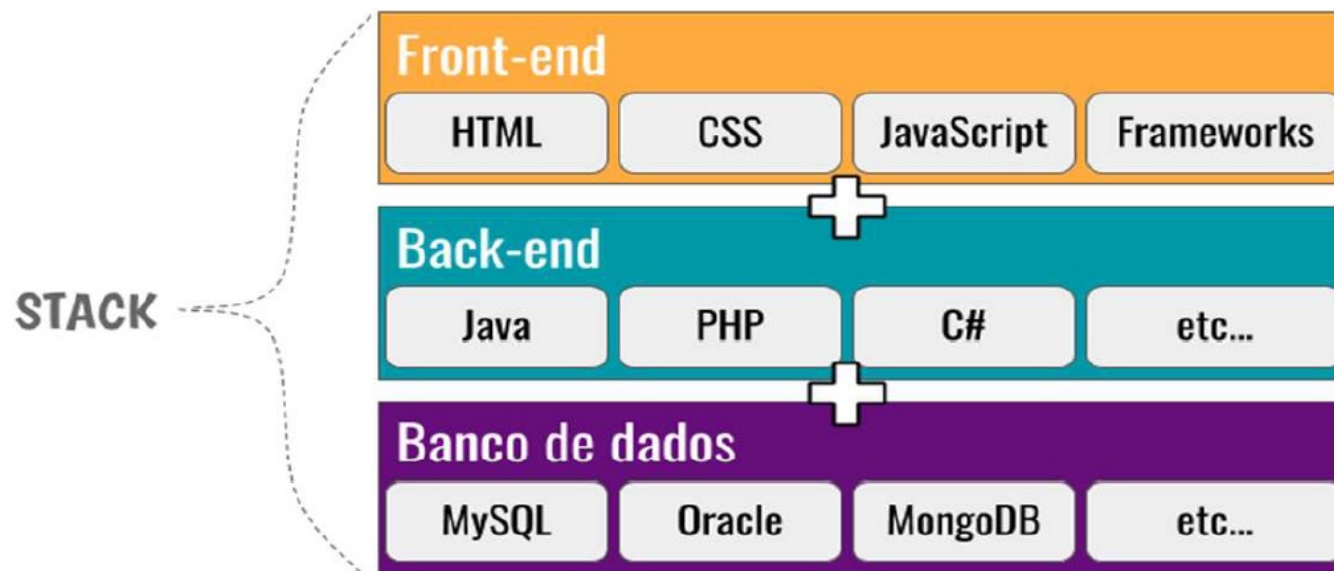
- **Programação Orientada à Objetos**

- Fundamentos
- Objetos, classes, atributos e métodos
- Abstração
- Encapsulamento
- Herança
- Relacionamento entre classes
- Polimorfismo
- Exceções
 - Definição
 - Tratamento

Programação Orientada a Objetos

Stacks

Conjunto de tecnologias que usamos para desenvolver nossos softwares.



Piores tipos de dores de cabeça

Enxaqueca



Hipertensão



Stress

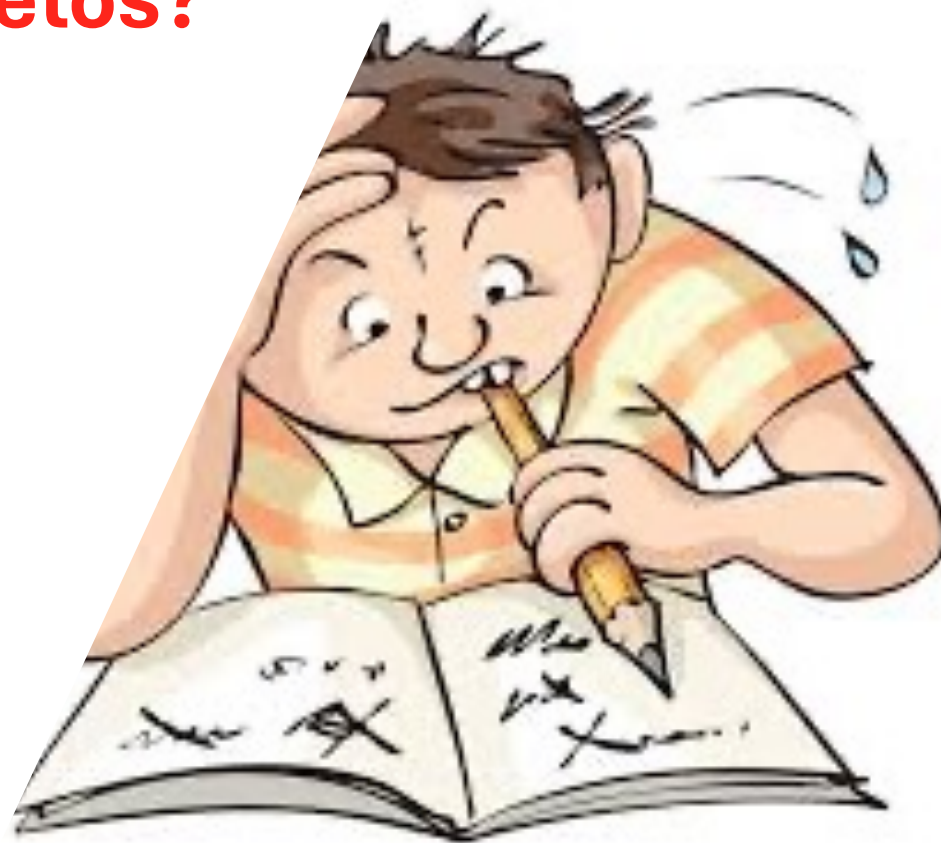


Java



O que é Orientação a Objetos?

Um paradigma de programação que permite representar programaticamente elementos do espaço de problemas.



Contextualizando o Problema

O que leva um programador a mudar do paradigma procedimental para um novo?

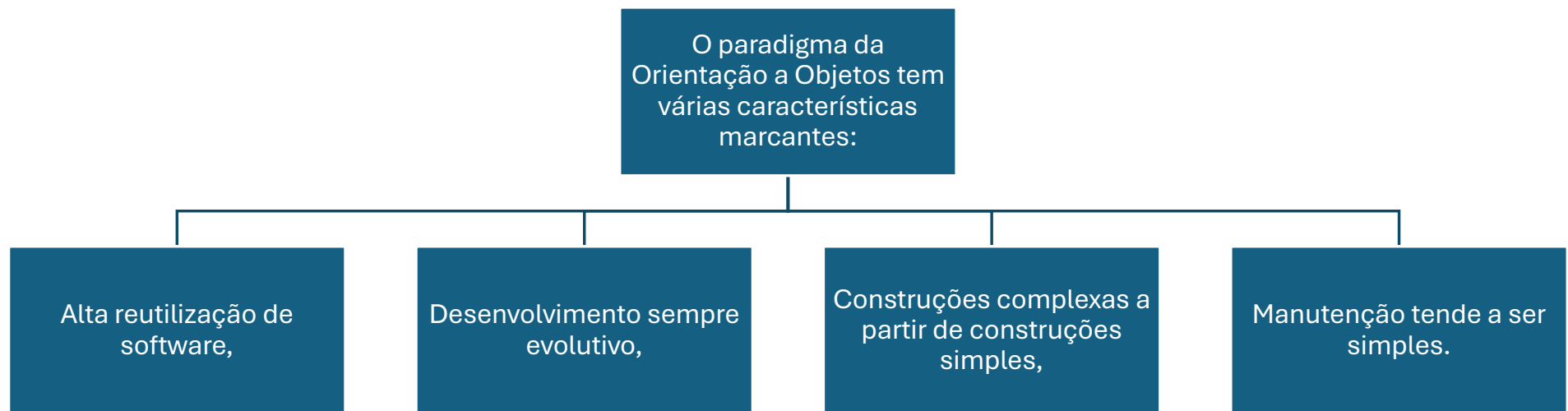
- Complexidade crescente dos sistemas
- Limitações da capacidade humana de compreensão de um sistema com um todo.

Paradigma

Em
Computação:

- Paradigmas explicam como os elementos que compõem um programa são organizados e como interagem entre si.

Orientação a Objetos

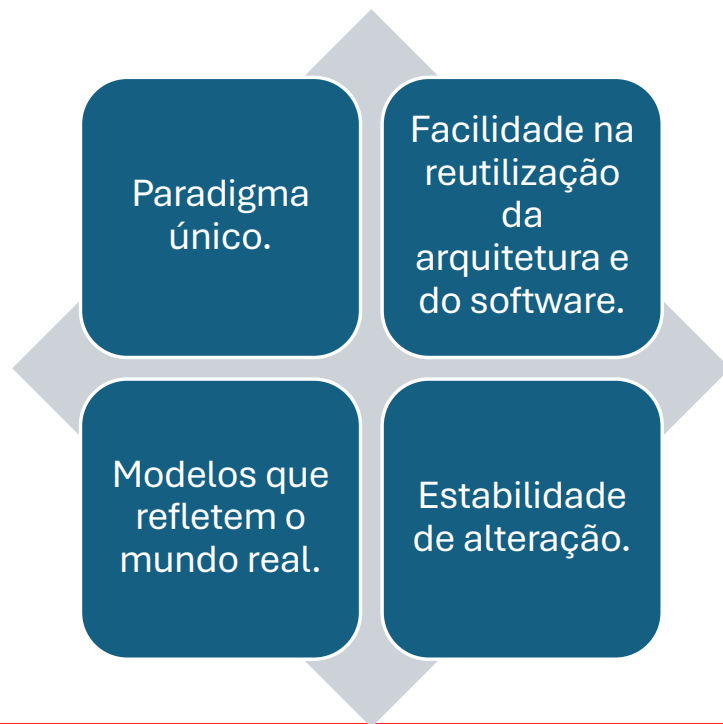


Por que Orientação a Objetos?

- A orientação a objetos garante, quando bem utilizada, oferecer ganhos em termos de rapidez, custo, confiabilidade, flexibilidade e facilidade de manutenção.



Por que Orientação a Objetos?



Paradigma único

A nomenclatura e os métodos utilizados na produção de um software orientado a objetos permanecem praticamente os mesmos.

Programadores, analistas, projetistas, usuários, etc. comunicam-se utilizando praticamente a mesma linguagem.

Facilidade de Reutilização

O software produzido é encapsulado em unidades básicas denominadas classes.

Essas classes possuem semânticas que podem ser reaproveitadas em novos softwares .

Facilidade de Reutilização

Exemplo:
Uma classe denominada
Funcionário
poderia ser
utilizada

- em um controle de departamento pessoal
- em um software de gerenciamento de distribuição de lucros anuais - PLR.

Modelos

Um modelo é uma representação adequadamente simplificada do mundo real

O modelo permite um entendimento facilitado e uma manutenção eficaz

Estabilidade de Alteração

- A manutenção e o impacto causados por alterações em requisitos e novas regras de negócio ficam bastante minimizados pelo uso de tecnologia de objetos.



O que é java?

- Java é uma especificação bem definida de uma Linguagem de Programação Orientada a Objetos, desenvolvida pela Sun Microsystems, e hoje de propriedade da Oracle.



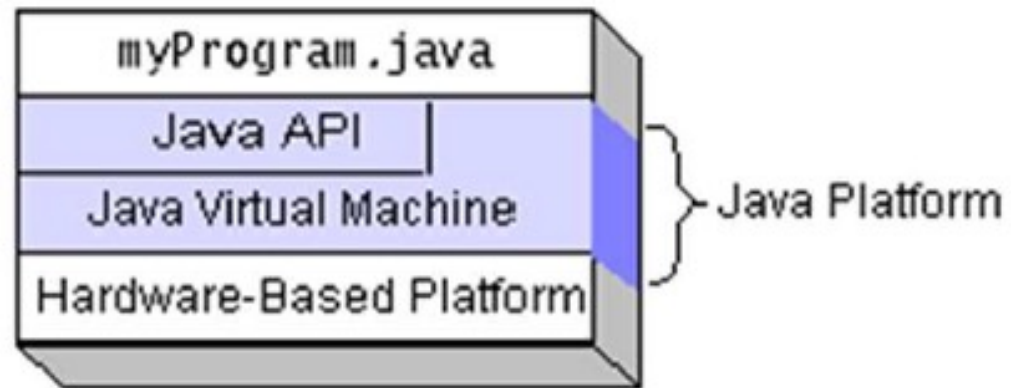
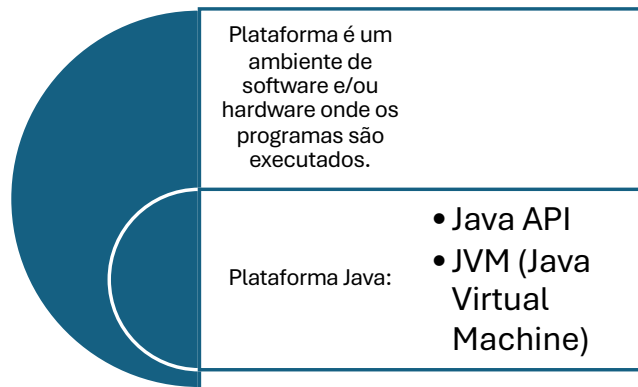
O que é java?

Usa-se,
ainda, o
nome Java
para
identificar:

- Ambientes de execução de software , presentes em SO, browsers , celulares, PDA, cartões inteligentes (JavaCards) e eletrodomésticos.
- Uma coleção de classes, componentes e frameworks (APIs) para o desenvolvimento de aplicações multiplataforma.
- O conjunto de ferramentas que permite o desenvolvimento de aplicativos Java (JDK – Java Development Kit)



O que é a Plataforma Java?



Vantagens da Linguagem Java

Ambiente de desenvolvimento OO

- OO é o paradigma mais forte atualmente
- Java é uma linguagem que atende os requisitos OO

Portabilidade do código produzido

- Programe em Windows e execute em Linux
- Sem necessidade de recompilar

Características de segurança

- Applets possuem restrições para acesso de arquivos
- Sem endereçamento direto de memória

Vantagens da Linguagem Java

Facilidade de integração a outros ambientes

- Java nasceu junto com a Internet
- API para integração com sistemas legados

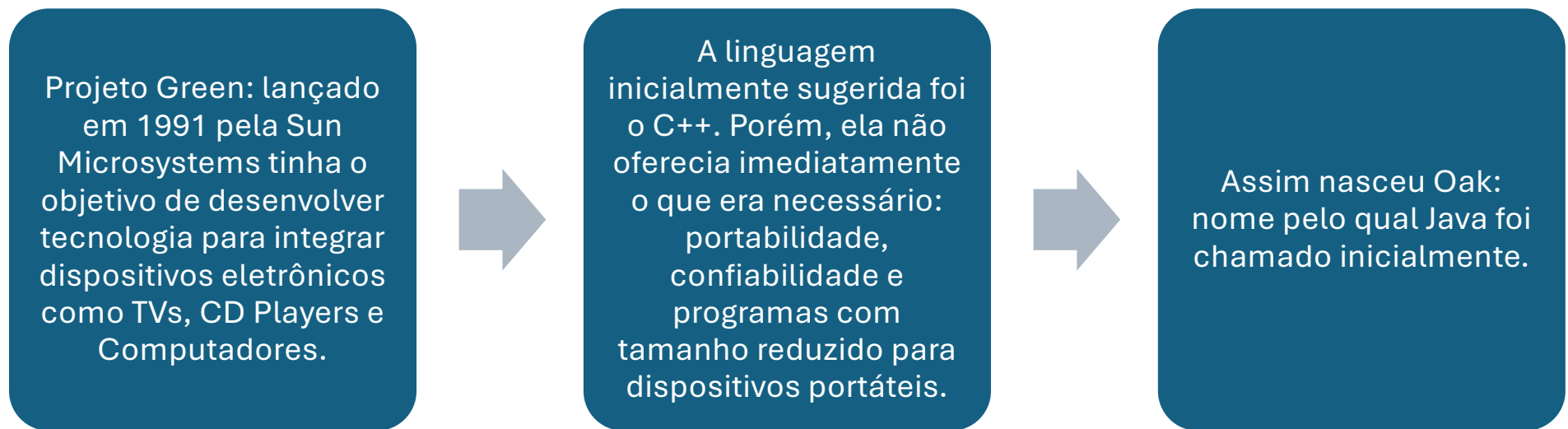
Linguagem e ferramentas em aperfeiçoamento

- Rumo da linguagem definido pela comunidade Java
- JCP (Java Community Process)

JDK é Free Software, assim como diversas ferramentas

- Java pode ser usado comercialmente sem nenhum custo
- Diversas ferramentas disponíveis gratuitamente

Como começou?



Como começou?

Oak foi criada pelo arquiteto e programador James Gosling, sendo uma mera ferramenta para os programadores do Projeto Green; mas acabou tendo um potencial maior do que qualquer um poderia ter imaginado.

Oak foi desenvolvida baseada em C++ e SmallTalk. Procurouse aproveitar as características positivas destas linguagens reavaliando os aspectos negativos.

Oak surgiu mais simples, segura, portátil e fácil de programar do que C++.

Como começou?

Em 1992 o Projeto Green lança um pequeno dispositivo portátil, um controle remoto extremamente inteligente.



Dotado de uma pequena tela com capacidade touch screen , o usuário interagia com eletrodomésticos através de uma representação animada de uma casa. Foi chamado Star-7.



Como começou?

Em 1993 surge a primeira grande oportunidade: uma concorrência pública para o desenvolvimento de uma tecnologia para TV a cabo interativa, porém vencida pela SGI (Silicon Graphics Inc.)



Nesta época a Internet era mais para aficionados em computadores que sabiam como manipular protocolos como FTP e Telnet.



Um programa chamado Mosaic, construído para o protocolo HTTP, tornou possível a usuários normais a visualização de documentos HTML na Internet. Este foi um marco importante.

Como começou?

Os membros do projeto perceberam que a tentativa de desenvolver uma interface interativa, robusta, segura e independente de arquitetura para dispositivos digitais também poderia ser usada na Internet!

A Internet era uma rede que ligava centenas de computadores diferentes executando sistemas operacionais diferentes.

Estimulados pelo grande crescimento da Internet, Jonathan Payne e Patrick Naughton desenvolvem o programa navegador WebRunner, capaz de efetuar o download e a execução de código Java via Internet.

Como começou?

Apresentado formalmente pela Sun como o navegador HotJava no SunWorld'95, o interesse pela solução se mostrou explosivo.

Assim se inicia a história de sucesso do Java.

Em setembro/1995, a Netscape Corp. lança a versão 2.0 de seu browser também com capaz de efetuar o download e a execução de pequenas aplicações Java, então denominadas applets.

- Applets são programas escritos em Java que podem ser adicionados em documentos hipertexto. Suportam efeitos de multimídia como sons, interações com o usuário (mouse, teclado), imagens, animações, gráficos.

Como começou?

Numa atitude importante, a Sun decide tornar o código-fonte aberto e disponibilizar o Java gratuitamente para a comunidade de desenvolvimento de software, embora detenha todos os direitos relativos à linguagem e as ferramentas de sua autoria.



Surge assim o JDK 1.0 (Java Development Kit 1.0)



Plataformas inicialmente atendidas: Sun Solaris e Microsoft Windows 95/NT. Progressivamente foram disponibilizados kits para outras plataformas tais como IBM OS/2, Linux e Macintosh.

Como começou?

Em 1997, surge o JDK 1.1 que incorpora grandes melhorias para o desenvolvimento de aplicações gráficas e distribuídas.

No início de 1999 é lançado o JDK 1.2, contendo tantas melhorias que foi denominado Java 2.

A Sun constantemente libera novas versões ou correções bem como novas APIs para desenvolvimento de aplicações específicas.

A versão estável mais atual disponível é o JDK 11.0.1 (LTS)

Características do Java

JCP (Java Community Process): processo criado pela Sun como forma de evoluir e manter a tecnologia Java aberta e disponível para todos.

Através do JCP são definidas as JSRs (Java Specification Request), que ditam as novas tecnologias Java.

Mais de 600 empresas participam:

- IBM
- Apache
- SAP
- Oracle
- Nokia
- Sony

Características do java

Simples e Dinâmica

- Java possui em torno de 50 palavras reservadas.
- As funcionalidades da plataforma estão nas APIs.

Orientada a Objetos

- Tudo em Java são classes ou instância de uma classe.
- Exceção: Tipos Primitivos.

Atende os requisitos para ser considerada OO:

- Abstração, Encapsulamento e Hereditariedade.

Características do Java

Portável (independência de plataforma)

- Java gera uma linguagem de máquina (bytecodes) destinada a uma única plataforma, a JVM (Java Virtual Machine).
- Em teoria, pode-se implementar uma JVM para qualquer plataforma de hardware.

Interpretada

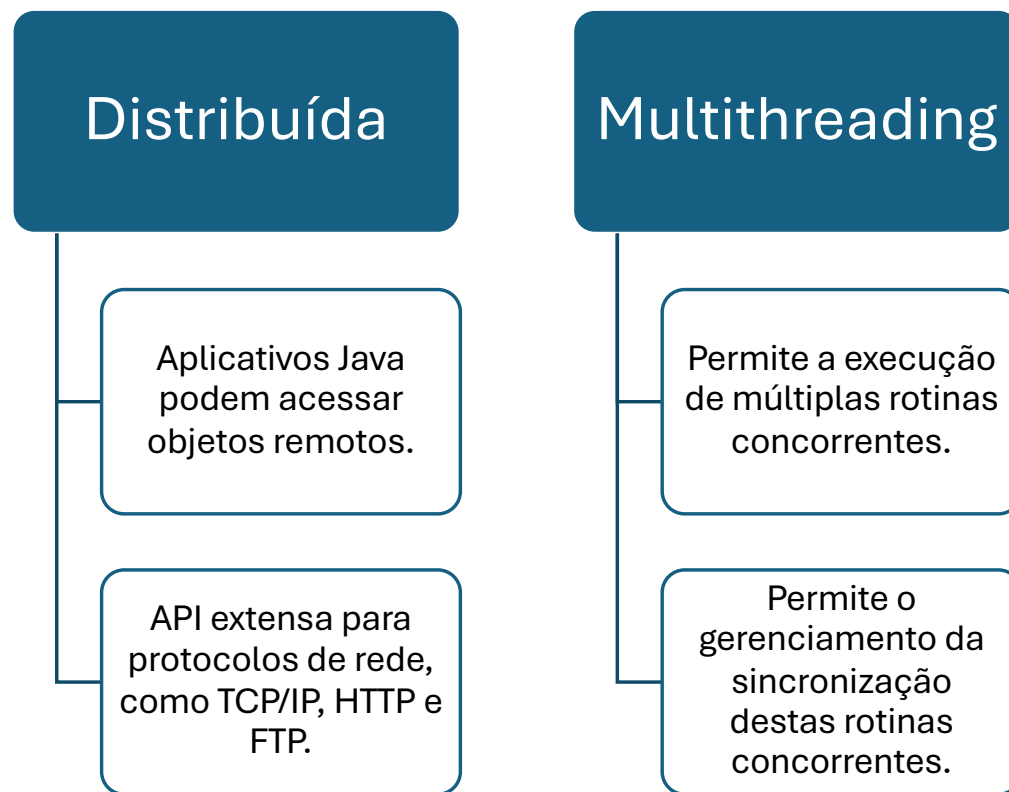
- Por ser interpretada não é comparada à velocidade de execução de código nativo.
- Algumas JVMs dispõem de compiladores JIT (Just In Time)

Características do Java

Robusta
e
Segura

- Não implementa aritmética de ponteiros.
- Possui mecanismo de liberação automática de memória.
- É fortemente tipada: não permite conversão implícita de tipos.
- Não permite herança múltipla de classes (mas possui interfaces).
- Exige o tratamento de exceções em tempo de compilação.
- Mecanismos de segurança podem, no caso de applets , evitar operações no sistema de arquivos.

Características do Java



Características do Java

Case-sensitive:

- Em Java maiúsculas de minúsculas são diferentes.

Não existem variáveis globais ou funções independentes:

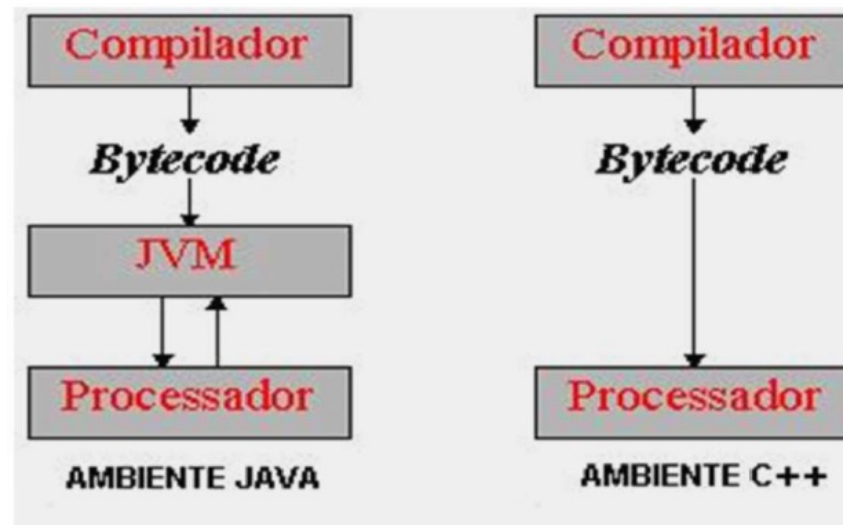
- Toda variável ou função pertence a uma classe ou objeto

Arquivo fonte tem extensão .java

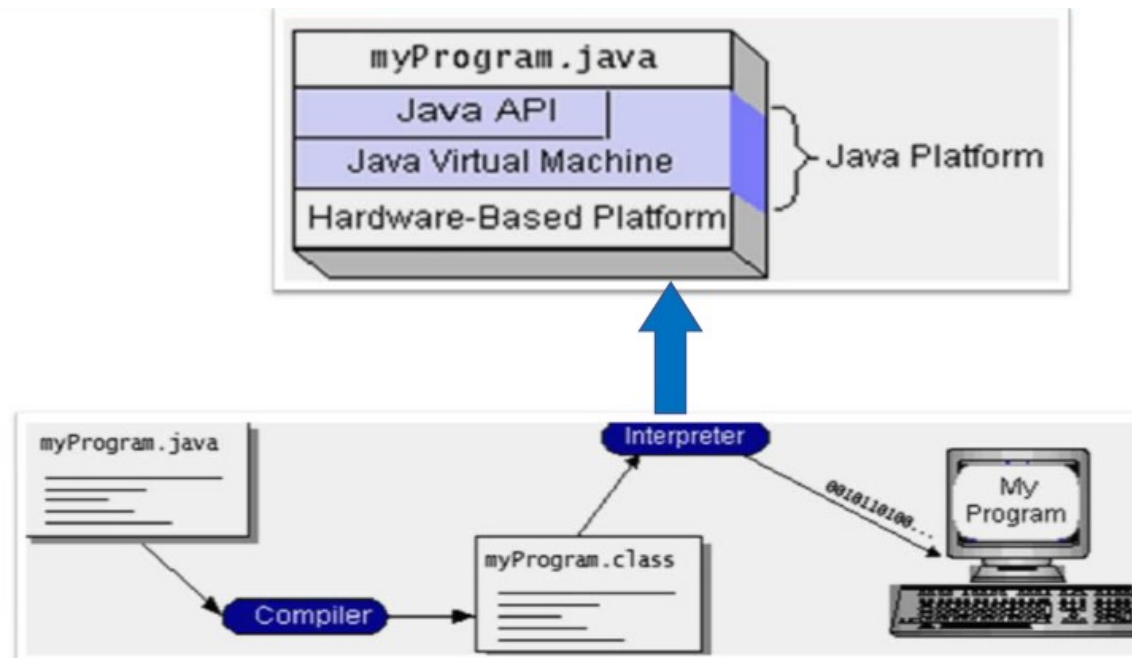
Classe compilada tem extensão .class

Classes podem ser agrupadas em arquivos .jar.

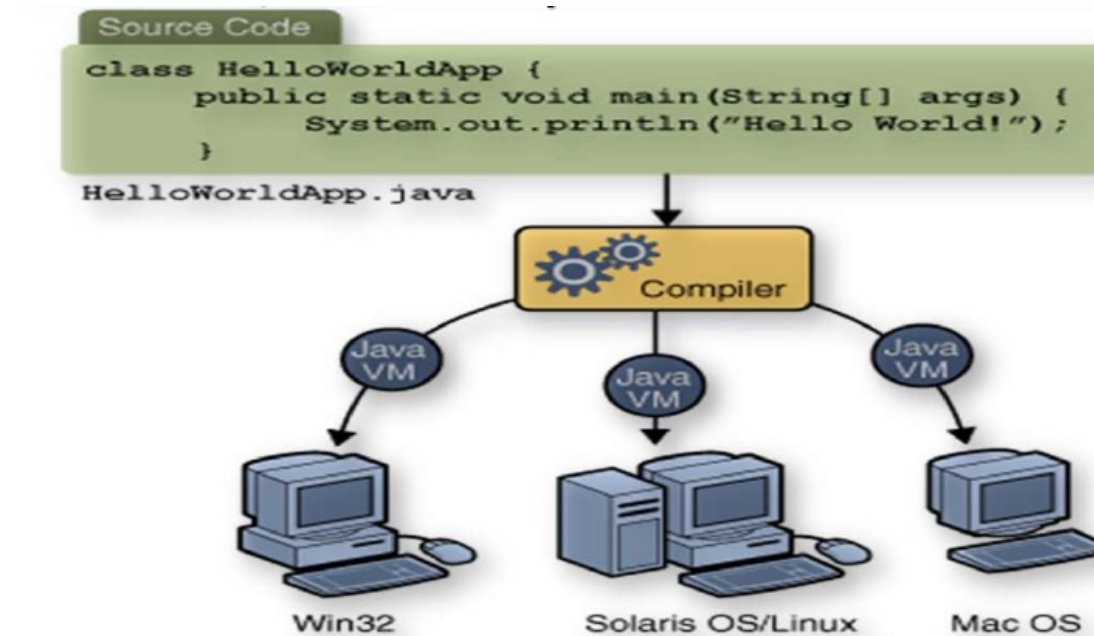
Java Virtual Machine



Java Virtual Machine



Write Once, Run Everywhere



Garbage Collector

Thread de baixa prioridade que libera da memória automaticamente objetos não mais referenciados no escopo do programa.

Reside na JVM.

Vantagem direta: libera o programador da responsabilidade de destruir objetos criados.



Distribuições Java

Java Standard Edition – Java SE

- APIs essenciais para qualquer aplicação java
- Core Java, Desktop Java

Java Enterprise Edition – Java EE

- APIs para o desenvolvimento de aplicações distribuídas
- JSP (Java Server Page), EJB (Enterprise Java Beans)

Java Micro Edition – Java ME

- APIs para o desenvolvimento de aplicações p/ portáteis
- PDA (Personal Digital Assistant), Palms, Celulares, TVs digitais

Palavras reservadas

Palavras-chave de Java

abstract	boolean	break	byte	case
catch	char	class	continue	default
do	double	else	extends	false
final	finally	float	for	if
implements	import	instanceof	int	interface
long	native	new	null	package
private	protected	public	return	short
static	super	switch	synchronized	this
throw	throws	transient	true	try
void	volatile	while		

APIs Java

Para trabalhar com Java é necessário conhecer suas APIs, as quais residem em pacotes (packages).

Cada distribuição possui uma coleção de APIs.

<u>Java Language</u>	Java Language					
<u>Tools & Tool APIs</u>	java	javac	javadoc	jar	javap	Scripting
	Security	Monitoring	JConsole	VisualVM	JMC	JFR
	JPDA	JVM TI	IDL	RMI	Java DB	Deployment
	Internationalization		Web Services		Troubleshooting	
<u>Deployment</u>	Java Web Start			Applet / Java Plug-in		
	JavaFX					
<u>User Interface Toolkits</u>	Swing		Java 2D	AWT	Accessibility	
	Drag and Drop	Input Methods		Image I/O	Print Service	Sound
<u>Integration Libraries</u>	IDL	JDBC	JNDI	RMI	RMI-IIOP	Scripting
<u>Other Base Libraries</u>	Beans	Security		Serialization		Extension Mechanism
	JMX	XML JAXP		Networking		Override Mechanism
	JNI	Date and Time		Input/Output		Internationalization
<u>lang and util Base Libraries</u>	lang and util					
	Math	Collections		Ref Objects		Regular Expressions
	Logging	Management		Instrumentation		Concurrency Utilities
	Reflection	Versioning		Preferences API		JAR Zip
<u>a Virtual Machine</u>	Java HotSpot Client and Server VM					

O que é aprender Java?

Conhecer a sintaxe da linguagem Java

- Tipos, controles, estruturas condicionais.

Aprender as APIs, suas classes e seus métodos

- `java.lang`; `java.io`; `java.util`; `java.sql`; `javax.swing`.

Conhecer a teoria da OO:

- Classes, Objetos, Métodos, Atributos, Herança.

Dominar ferramentas e tecnologias auxiliares

- IDEs, UML, XML, Design Patterns, etc.

Como começar com Java?

Instalar JDK.

- Download em [oracle.com](https://www.oracle.com)
- JDK - não é ambiente visual (IDE), é um conjunto de ferramentas para desenvolvimento Java.

Composto por:

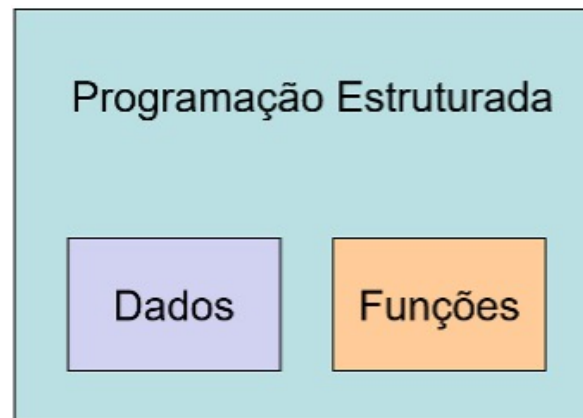
- javac.exe – Compilador Java
- java.exe – Máquina virtual Java (JVM)
- jre.exe – Java Runtime Environment
- appletviewer.exe – Visualizador de Applets
- javadoc.exe - Composição de documentação
- jdb.exe – Depurador de programas

Um pouco da história...



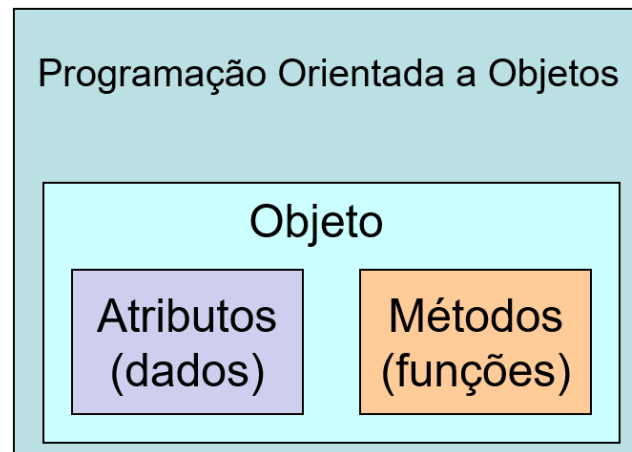
Programação Estruturada

Quando falamos em programação estruturada nos baseamos em sistemas top-down, ou seja, programação de cima pra baixo, dividindo a lógica de programação em dados e funcionalidades. Nas aplicações estruturadas os dados ficam separados das funcionalidades.



Programação Orientada a Objetos

Já na programação orientada a objetos, os dados ficam armazenados em cada objeto existente. De forma mais genérica um objeto é algo que pode ser representado e apresenta algum significado ou sentido



Orientação a Objetos

A orientação a objetos tem por objetivo representar, de forma idêntica as situações do mundo real em sistemas computacionais, portanto os sistemas operacionais não devem ser vistos como um conjunto de estruturas de processos e sim como uma coleção de objetos do mundo real que interagem uns com os outros de forma organização e sistêmica.

Orientação a Objetos

A orientação a objetos tem por objetivo representar, de forma idêntica as situações do mundo real em sistemas computacionais, portanto os sistemas operacionais não devem ser vistos como um conjunto de estruturas de processos e sim como uma coleção de objetos do mundo real que interagem uns com os outros de forma organização e sistêmica. A orientação a objetos possui os seguintes pilares:



Vantagens da Orientação a Objetos

- **Organização:** por conta da modularidade há aumento do nível de organização e simplicidade do código.
- **Produtividade:** por conta do reaproveitamento do código, pois quando um desenvolvedor cria um objeto para realizar uma tarefa complexa os demais desenvolvedores que precisam interagir com esse objeto precisam apenas acessá-lo para executar a mesma tarefa.
- **Redução do custo:** após a criação de uma estrutura eficaz de objetos é possível incluir módulos adicionais ao sistema que utilizam funcionalidades já desenvolvidas, portanto utilização do mesmo código em sistemas diferentes.
- **Reaproveitamento:** pode-se transformar objetos semelhantes para utilização em novos sistemas.
- **Facilidade de manutenção:** quando é necessário realizar uma alteração no sistema é possível que o desenvolvedor adapte, exclua ou inclua novos objetos ao sistema rapidamente, sem comprometer o funcionamento do mesmo.
- **Trabalho em equipe:** por conta da modularidade é fácil dividir as tarefas em diversas equipes de programação.

O que é um objeto?

- Enquanto a programação estruturada baseia-se em ações, a programação orientada a objetos baseia-se em **OBJETOS**;
- Um objeto representa “uma coisa”, assim como na vida real;
 - E, da mesma forma, podem ser simples ou complexos;
 - Uma pequena bola de tênis é um objeto, assim como a Starship também!



Objetos

- A maneira como definimos os objetos que usamos depende do nível de detalhes que precisamos no contexto de seu uso;
- Todo objeto sempre terá um **Nome**;
- E pode conter ou fazer referência a outros objetos;
 - Por exemplo: Comprador e Nota Fiscal
 - Uma relação de consumerista, o comprador possui uma nota fiscal com produtos, quantidades, valores, dados etc.

Objetos

Podemos descrever os objetos usando suas propriedades:

- Cor, peso, preço, velocidade, código de barras, fabricante (...)

Exemplo:

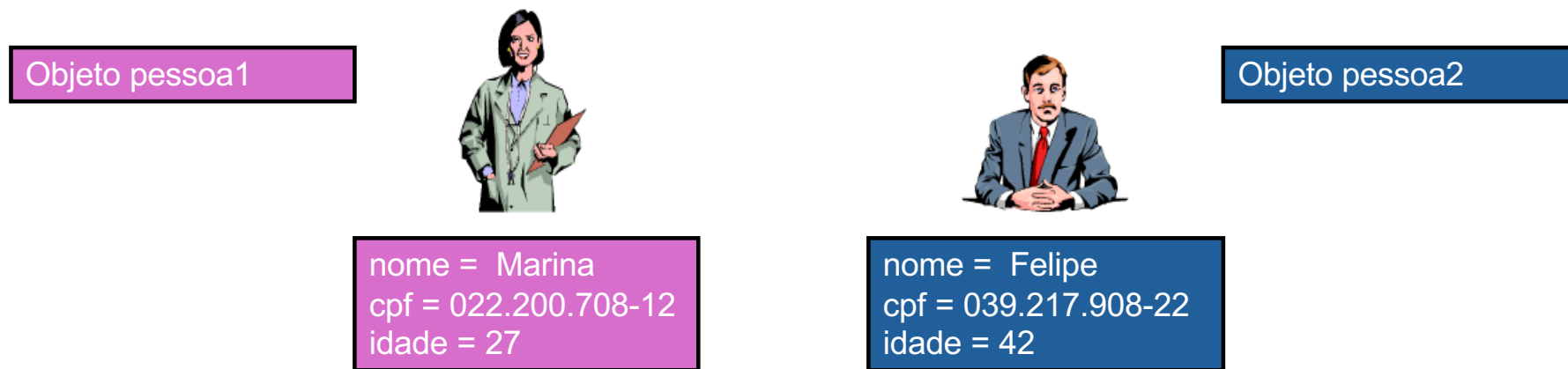
- Nossa bola de tênis pode ter uma cor, um peso, um preço e um fabricante em um determinado contexto;

Objetos no mundo real e na programação podem ter um estado. A alteração do estado de um objeto não afeta outro objeto.

Estado do objeto???

O que é o estado do objeto?

- Condição atual do objeto, a cor atual, a posição, o valor atual etc.



Ambos são objetos de uma classe, provavelmente uma classe chamada Pessoa, Cliente, Funcionário, etc... Mas e agora... **o que são classes???**

Classes

As Classes e os Objetos possuem uma relação de dependência, pois não existe objeto sem a classe. A Classe representa a abstração das características comuns mais relevantes (atributos e métodos) de um conjunto de objetos. A classe passa a ser um molde para se criar objetos do mesmo tipo.

Classes

Para construir os objetos, devemos estabelecer quais serão suas características (atributos) e seus comportamentos (métodos). Inúmeros objetos podem ser construídos a partir das classes.



Podemos dizer que cada um desses desenhos são um “objeto” e que foram gerados através da classe “Veículo”.

Quais as características dos objetos da classe veículo?

Quais os comportamentos dos objetos da classe veículo?

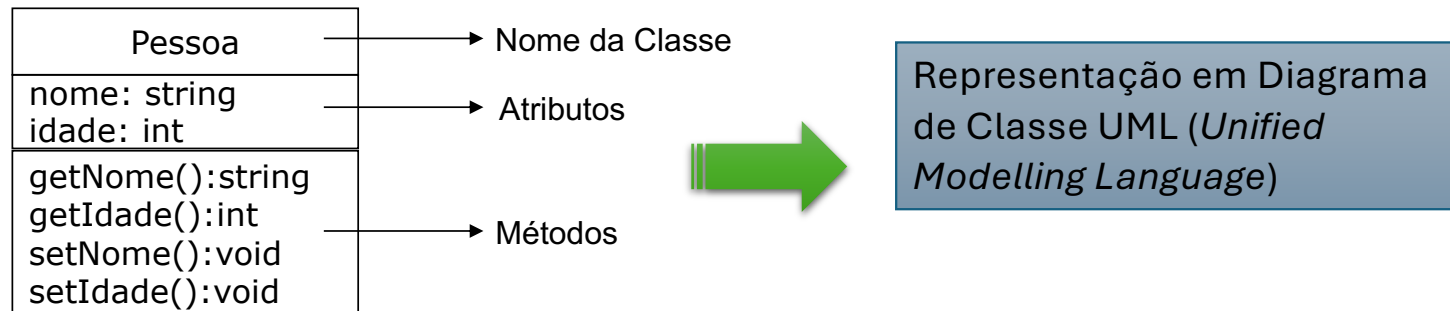
Classes

- Na programação, não criamos objetos, mas sim classes;
- Precisamos de classes para criarmos objetos;
- A classe é como o “projeto” do objeto;
- Pense na classe como um plano, uma descrição do que será cada objeto;

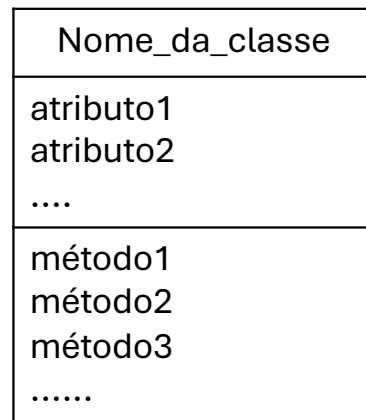
Classes

Exemplo:

A classe Pessoa deverá ter atributos e métodos comuns



Construindo Classes em Java



Modelo UML de classe



```
public class <Nome_da_classe> {  
  
    <lista de atributos>  
  
    <lista de métodos>  
  
}
```



Um arquivo em Java precisa
ser uma classe pública com o
mesmo nome do arquivo

Classes

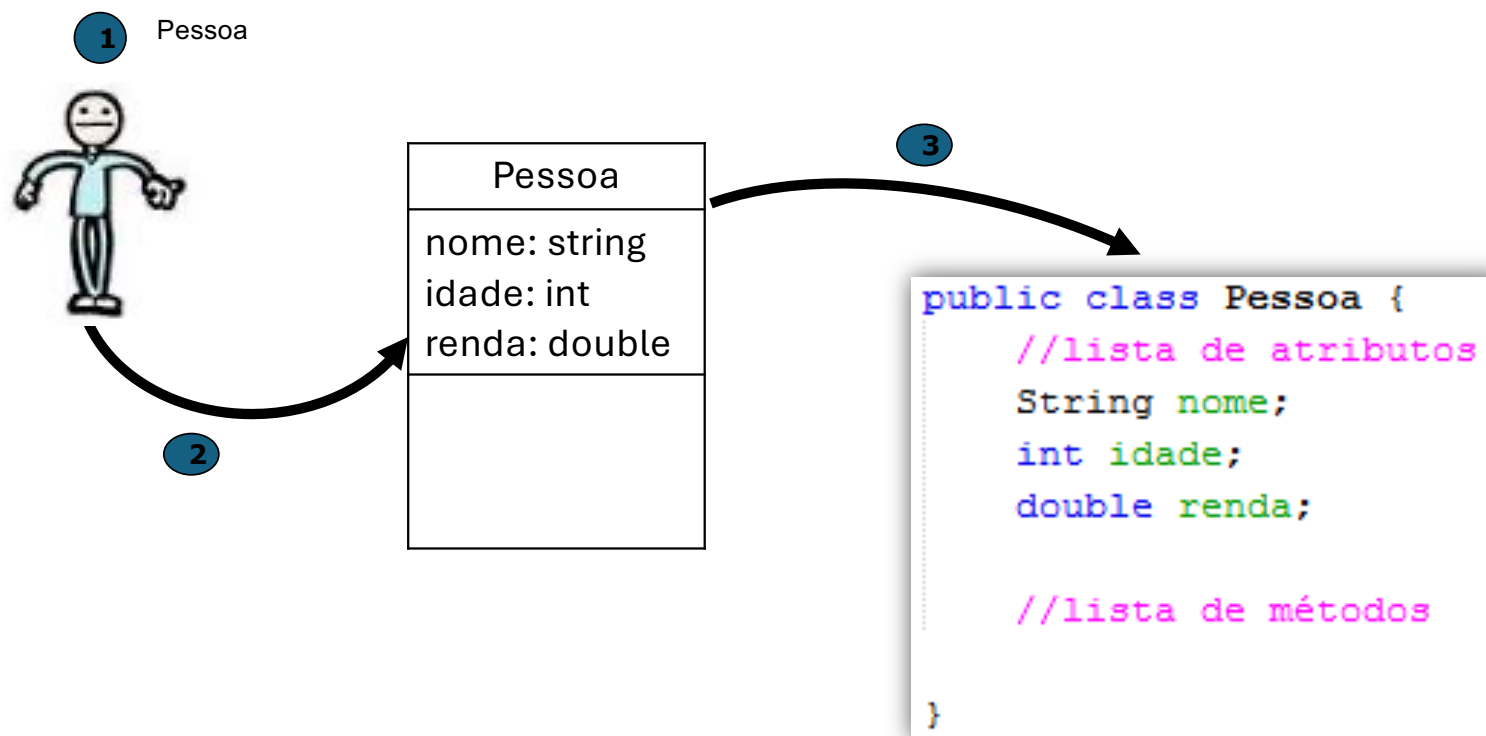
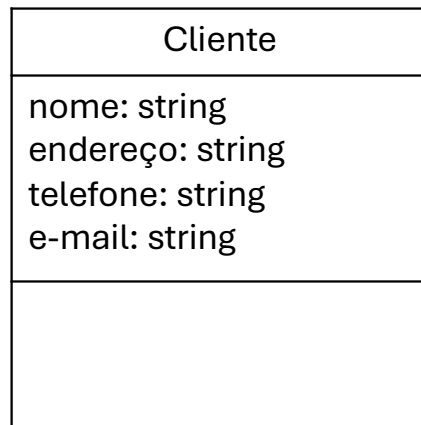


Diagrama UML

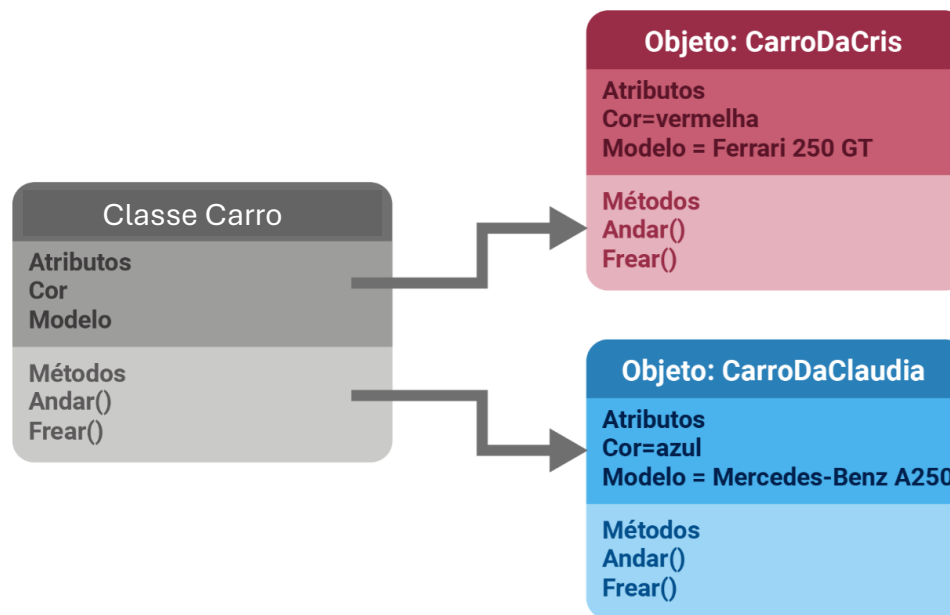


Classe em Java

```
public class Cliente {  
    //atributos  
    String nome;  
    String endereco;  
    String telefone;  
    String email;  
  
    //lista de métodos  
  
}
```

Atributos

Podemos definir atributos como valores de dados assumidos pelos objetos de uma classe, portanto, para cada instância de uma classe o atributo possui um valor, que pode ser o mesmo para instâncias diferentes (FELIX, 2016).



Atributos

Os atributos de um objeto só podem ser alterados através de eventos que provoquem a sua mudança de estado, sendo que cada objeto deve ser responsável pela mudança de seus próprios atributos e nenhum objeto deve possuir a capacidade para interferir nos atributos de outro objeto.

Essa frase um pouco mais adiante fará total sentido para você...

Exemplo

Digamos que você identificou, em um projeto de um jogo que está desenvolvendo, a necessidade de ter o objeto “**Chefão**”.

Este objeto possui características como: **nome, conjunto de poderes, armas e level.**

Assim como possui comportamentos de **atirar, correr, defender e morrer.**

Repare que a classe é somente um “*modelo*” do objeto:

- Chefão
 - Nome, poderes, level
 - atirar, correr, defender, morrer

Não colocamos os dados nas classes, somente nos objetos que elas nos permitem criar.

Exemplo

Chefão



Nome

Nome, poderes, level.



Atributos ... “variáveis”

atirar, correr, defender, morrer.



Métodos ... “funções”

Exemplo

Classe

- Chefão
 - Nome, poderes, level
 - atirar, correr, defender, morrer

Objeto 1: Chefão

Nome: Bowser
Poderes: fogo, bomba
Level: 2

Objeto 2: Chefão

Nome: Shao Kahn
Poderes: chute, soco
Level: 6

Objeto 3: Chefão

Nome: Dr. Wily
Poderes: voo, bomba
Level: 3

Exemplo

Classe

- Chefão
 - Nome, poderes, level
 - atirar, correr, defender, morrer

Todos os objetos dessa classe
compartilham os mesmos métodos
(comportamentos)

Objeto 1: Chefão

Nome: Bowser
Poderes: fogo, bomba
Level: 2

Objeto 2: Chefão

Nome: Shao Kahn
Poderes: chute, soco
Level: 6

Objeto 3: Chefão

Nome: Dr. Wily
Poderes: voo, bomba
Level: 3

Mais sobre classes...

- Podemos criar quantos objetos forem necessários com uma classe;
 - Naturalmente que todos terão o mesmo conjunto de atributos e comportamentos
- Outro grande benefício do uso de classes é o fato de podermos empacotá-las em bibliotecas ou estruturas para reutilização em outros projetos, projetos derivados etc;
- Todas as linguagens de programação modernas (Java, Go, Dart, JS, PHP, C#, C++, Python...) permitem ou utilizam orientação a objetos;
 - E permitem o uso de todas essas funcionalidades descritas até então
- Com isso não precisamos reinventar a roda reimplementando funcionalidades que já estão disponíveis!

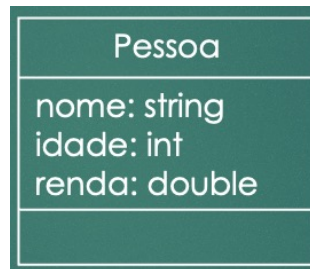
Pra finalizar, e instância, o que é?

A classe é um modelo para os objetos do sistema, desta forma após a definição da classe podemos **instanciar** (criar) objetos que serão do tipo da classe. Eles são chamados de instâncias.

Portanto, cada objeto é uma **instância** de uma classe.



```
Pessoa: pessoa1  
nome = Marina  
renda = 3500,00  
idade = 27
```



```
Pessoa: pessoa2  
nome = Felipe  
renda = 1200,00  
idade = 42
```

Instanciando objetos

Classe (iniciais em maiúsculas)

```
public class Pessoa {  
    //lista de atributos  
    String nome;  
    int idade;  
    double renda;  
}
```

Atributos
(em letras minúsculas)

```
public class UsaClasses {  
  
    public static void main(String[] args) {  
        //Pessoa p = new Pessoa();  
        Pessoa p;  
        p = new Pessoa();  
        Pessoa p1 = new Pessoa();  
  
        p.nome = "Fulano";  
        p.idade = 25;  
        p.renda = 1000;  
  
        System.out.println("Nome: " + p.nome);  
        System.out.println("Idade: " + p.idade);  
        System.out.println("Renda: " + p.renda);  
  
        System.out.println("Nome: " + p1.nome);  
        System.out.println("Idade: " + p1.idade);  
        System.out.println("Renda: " + p1.renda);  
    }  
}
```

Objetos (em letras minúsculas)

Instanciando objetos

```
public class UsaClasses {
```

```
    public static void main(String[] args) {
```

```
        //Pessoa p = new Pessoa();
```

```
        Pessoa p;
```

```
        p = new Pessoa();
```

```
        Pessoa p1 = new Pessoa();
```

```
        p.nome = "Fulano";
```

```
        p.idade = 25;
```

```
        p.renda = 1000;
```

```
        System.out.println("Nome: " + p.nome);
```

```
        System.out.println("Idade: " + p.idade);
```

```
        System.out.println("Renda: " + p.renda);
```

```
        System.out.println("Nome: " + p1.nome);
```

```
        System.out.println("Idade: " + p1.idade);
```

```
        System.out.println("Renda: " + p1.renda);
```

```
    }
```

```
}
```

Método principal por onde o Java começa a execução do programa

Instanciando 2 objetos da classe Pessoa. Cada um com o seu identificador

Estamos criando o estado do objeto p1, acessando os seus atributos através de um ponto após o nome do objeto

Podemos também acessar os atributos e recuperar o estado dos objetos para imprimir na tela o seu conteúdo

Outros Exemplos

Triangulo
base: float altura: float

Data
dia: int mes: int ano: int

Curso
nome: string qtdealunos: int turma: string

Exercício 01

Crie um programa em Java composto por duas classes. A primeira classe deve se chamar Carro e conter três atributos: modelo, ano e preço. Além disso, a classe deve possuir dois métodos: um método chamado aplicarDesconto, que recebe um valor percentual e retorna o preço do carro com o desconto aplicado, e um método chamado exibirInformacoes, que imprime no console o modelo, o ano e o preço do carro. A segunda classe deve se chamar UsaClasses e conter o método principal (main). Dentro desse método, crie três objetos da classe Carro, atribuindo valores fixos aos seus atributos. Em seguida, utilize o método aplicarDesconto para simular uma promoção em pelo menos um dos carros. Por fim, exiba no console as informações de todos os carros chamando o método exibirInformacoes.

Exercício 02

Crie um programa em Java composto por duas classes. A primeira classe deve se chamar Aluno e conter cinco atributos: nome, curso, ano de ingresso, número de disciplinas cursadas e média geral. Além disso, a classe deve possuir dois métodos: o primeiro deve se chamar calcularSemestresConcluidos e retornar quantos semestres o aluno já cursou, considerando que cada ano possui dois semestres e que o ano atual é 2025; o segundo deve se chamar aprovado e retornar verdadeiro se a média geral for maior ou igual a 6.0, e falso caso contrário. A segunda classe deve se chamar UsaClasses e conter o método principal (main). Dentro desse método, crie três objetos da classe Aluno, atribuindo valores fixos aos seus atributos. Em seguida, utilize os métodos criados para mostrar no console, junto das informações de cada aluno, quantos semestres ele já concluiu e se está aprovado ou não.

Já sei tudo?

- Só isso? Já sei tudo que diz respeito à orientação a objetos?
- Não, ainda há um longo caminho pela frente, com muitos conceitos importantes ;
 - Abstração, Encapsulamento, Herança, Polimorfismo etc.
- Agora que tudo começa a ficar realmente divertido! 😬 😊 (é sério!)





That's all Folks!